

MAE 423: Introduction to Propulsion
Final Project

**Design and Optimization of Single Stage
Sounding Rockets**

Written By:
Humza Khan
(50363412)

Dated:
05/05/2025

Professor:
Dr. Paul DesJardin

Teaching Assistant:
Jason Sun

Problem Statement:

This report investigates the design and optimization of a single stage sounding rocket under varying mission parameters and constraints that were provided in the project overview document as shown in Table 1. A ‘medium’ option of the parameters was used after the selection of any one parameter, ensuring that only one design specification is changed at a time. This resulted in seven unique cases as shown in Table 2 whose results can clearly distinguish what effects changing certain design parameters can have on the rocket’s performance. Design Specifications like ρ_s , ρ_p , σ_s , SM , and N remained constant between the seven different cases, while Payload Mass (M_L), Max Altitude (h_{max}), and Normalized Acceleration (a_{max}) were varied based on the aforementioned methodology.

Using the 8 inputs for each case, a MATLAB script was written such that it calculates all possible combinations between a range of basic rocket geometry (parameterized in terms of L and D) and outputs the needed variables. After all possible outputs were calculated, the L and D values that delivered the highest payload ratio (λ) were extracted for each case, along with the resulting X_{CP} and X_{CG} locations.

The extracted geometry data for the baseline case (*all medium options*) was used to model a rocket in the OpenRocket software, where after entering the correct parameters and material selections the predicted X_{CG} and X_{CP} locations were compared with what was outputted by the MATLAB script. If the predicted locations from OpenRocket were within 1 – 2% of the MATLAB script predictions, the result was deemed to be true or verified.

Table 1: Design Input Parameters

Design Input Parameters		
Design Spec.	Value	Description
$M_L(kg)$	5, 10, 20	Payload mass
$h_{max}(ft)$	10k, 20k, 30k	Maximum altitude
a_{max}^*	5, 10, 20	Normalized max acceleration
SM	2	static margin = $(X_{CP} - X_{CG})/D$
$\rho_s(kg/m^3)$	2700	shell density (aluminum)
$\rho_p(kg/m^3)$	1772	propellant density, 66 – 78% AP, 18% organic polymer, 4 – 20% Al
$\sigma_s(MPa)$	150	shell working stress
N	4	number of fins

Table 2: The Seven Cases and the Varied Inputs

	M_L (kg)	h_{max} (m)	a_{max}
Case 1	5	6095.7025	10
Case 2	20	6095.7025	10
Case 3	10	3047.8513	10
Case 4	10	9143.5538	10
Case 5	10	6095.7025	5
Case 6	10	6095.7025	20
Case 7	10	6095.7025	10

Method of Solution:

In order to obtain appropriate results, a MATLAB script (Appendix A) was written that takes the data from table 2 as a [7x3] matrix as input and iterates it with other design specifications in a FOR loop to calculate optimal mass ratio (R_{opt}), equivalent velocity (W_{eq}), burn time (t_b), and Mach number (M_{eq}) for each case via Eq. (1), Eq. (2), Eq. (3), Eq. (4) respectively.

$$R_{opt} = 1 + a *_{max} \quad (1)$$

$$W_{eq} = \sqrt{\frac{h_{max}}{\frac{1}{2} \log(R_{opt}) (\log(R_{opt}) - 2) + \frac{R_{opt} - 1}{R_{opt}}}} \quad (2)$$

$$t_b = \frac{(R_{opt} - W_{eq}) - 1}{g R_{opt}} \quad (3)$$

$$M_{eq} = \frac{W_{eq}}{a_0} \quad (4)$$

After these variables were solved for in the loop, ideal isentropic relations were used to derive values for the stagnation temperature (T_0) and pressure (P_0) for each case. Once these values were obtained, the variables for D and L (geometry parameters) were initialized with D ranging from 0 to 1 meter in 5000 increments, and L ranging from 0 to 5 meters in 5000 increments. These variables were then combined in a [5000x5000] matrix that contains all the possible combinations that exist in the ranges of D and L .

Proceeding further, the second FOR loop initiates and uses D , P_0 , and σ_s to solve for the required wall thickness (δ) for each increment of D via Eq. (5). Progressing in the same loop, the shell density (ρ_s), the surface area of the geometries, and the wall thickness is used to calculate the mass of each component of the rocket. The component masses are then added to obtain the total mass of the structure (M_s) for that combination of D and L using Eq. (6).

$$\delta = (DP_0)/(2\sigma_s) \quad (5)$$

$$M_s = M_{cone} + M_{body} + M_{fins} \quad (6)$$

Carrying onwards from the obtained values, the required mass of the propellant (M_P) is calculated for that iteration in the loop as shown in Eq. (7). The result from Eq. (7) is incorporated in Eq. (8) along with the density of the propellant (ρ_p) and the D value at the iteration to calculate the length of the propellant at that increment. All the obtained masses and the payload mass are combined to calculate the total mass of the rocket (M_O) using Eq. (9).

$$M_P = (R_{opt} - 1)(M_s + M_L) \quad (7)$$

$$L_P = (4M_P)/(\pi D^2 \rho_p) \quad (8)$$

$$M_O = M_s + M_P + M_L \quad (9)$$

After this, the X_{CG} locations for each component of the rocket were solved with respect to their geometries and shapes. Combining these components and their respective masses, the location of the CG (center of gravity) of the rocket at that iteration was calculated via Eq. (10).

$$X_{CG} = \left(\frac{(X_{CG,fins})(M_{fins}) + (X_{CG,tube})(M_{tube}) + (X_{CG,P})(M_P) + (X_{CG,Cone})(M_{cone}) + (X_{CG,Cone})(M_L)}{M_O} \right) \quad (10)$$

The loop then progresses to solve the location of the CP (center of pressure) of all the components of the rocket at that increment of D and L . The equations listed in the provided “Centuri Report” document were used to solve for the variables like k_{fb} , C_{na_f} , $C_{na_{fb}}$, C_{na_n} , x_f , x_{f_bar} , and δx_f , based on the fin geometry at that iteration. Using those variables, along with the X_n of the nose cone, the location of the CP of the rocket (X_{CP}) was calculated using Eq. (11).

$$X_{CP} = \frac{((C_{Na})_n X_{ncone} + (C_{Na})_{fb} \bar{x}_f)}{C_{Na}} \quad (11)$$

Finally, the payload ratio (λ) is calculated via Eq. (12) and the highest λ for each case is stored. Constraints involving the static margin (SM) equation mentioned in Table 1 are enforced and structural ratio (ε) of the is calculated using Eq. (13) based on the D and L values that result in the highest λ . The rest of the required outputs that were solved earlier are similarly extracted from their respective matrices based on the D and L values that yield the highest λ and the results in Table 3 are compiled in their respective cases.

$$\lambda = M_L/(M_s + M_P) \quad (12)$$

$$\varepsilon = M_s/(M_s + M_P) \quad (13)$$

Discussion of Results

MATLAB Optimization Results for Each Case

Table 3: The Results from the MATLAB script

	R_{opt}	W_{eq} (m/s)	T_{burn} (s)	P_o/P_a	δ/D	D (m)	L/D	X_{CG} (m)	X_{CP} (m)	M_p (kg)	M_s (kg)	M_o (kg)	λ_{max}	ε
Case 1	11	207.7025	19.2478	1.2756	4.31E-04	0.4981	9.33333	5.15481	4.15861	102.067	5.20669	112.274	0.04661	0.04854
Case 2	11	207.7025	19.2478	1.2756	4.31E-04	0.26625	6.31856	1.08389	1.61639	205.873	0.58729	226.46	0.09687	0.00284
Case 3	11	146.8679	13.6102	1.13182	3.82E-04	0.18564	17.5808	2.33551	2.70678	104.102	0.41023	114.513	0.09568	0.00393
Case 4	11	254.3826	23.5736	1.43198	4.84E-04	0.20984	6.91611	0.94899	1.36868	103.454	0.3454	113.799	0.09634	0.00333
Case 5	6	304.0674	25.8297	1.65268	5.58E-04	0.17924	1.85268	0.1248	0.48328	50.5518	0.11037	60.6622	0.19739	0.00218
Case 6	21	153.3638	14.889	1.14431	3.86E-04	0.22364	20.0089	3.22402	3.67131	216.286	0.81428	227.1	0.04606	0.00375
Case 7	11	207.7025	19.2478	1.2756	4.31E-04	0.19424	13.2389	1.80645	2.19493	104.134	0.41336	114.547	0.09565	0.00395

The results in Table 3 were compiled after the MATLAB script had run through all the possible combinations of D and L parameters. The values that yielded the highest λ for each case were extracted. Case 7 was set as the base case with all the medium options for M_L , h_{max} , and a_{max} selected. This allowed the other cases to vary only one variable from the base case and this simplified the analysis in terms of the resulting changes.

1. **Varying M_L :** It can be observed from Table 3 that as the payload mass is reduced by 50% in Case 1, the rocket needs to be a lot more structurally heavier than the base case (+1168%) while maintaining the same SM in order to perform similarly to Case 7. The D parameter is increased significantly (+156%), suggesting that the L dimension has also increased based on the L/D ratio. The resulting λ for Case 1 is around 50% of Case 7's but due to increased M_s , ε is significantly increased as well. Somewhat opposite changes are noticed in Case 2, where M_L is increased by 100%. Case 2 rocket only has an increase of 42% in M_s as compared to the base case, which can be explained by the increased D parameter. The total mass (M_o) has increased by about 98% as more propellant (M_p increased by ~100%) is needed to launch the additional payload. The maximum λ for Case 2 is slightly varied by around 1% from Case 7 while ε is reduced by ~28%.
2. **Varying h_{max} :** In Case 3, the maximum apogee of the is reduced by 50% from the base case. Although the propellant mass is roughly the same as Case 7, the burn time (t_b) has been reduced by about 32% in Case 3. Since the rocket needs to cover about half the original height, the equivalent velocity is also reduced in Case 3 when compared to Case 7. The D dimension is reduced by about 6%, the increased L/D ratio suggests a taller configuration while maintaining the same SM for stability. The wall thickness for Case 3 is has also reduced by 11% while λ and ε remain roughly the same. However, increasing the apogee by 100% from the base case in Case 4 results in an increased W_{eq} and t_b . While D increases by roughly 2 cm in Case 4, the rocket is shorter in length as compared to Case 7 as suggested by the reduced L/D ratio. Case 4's λ increases by about 1% while ε reduces by about 15% when compared to the base case.
3. **Varying a_{max} :** It can be observed in Table 3 that when the a_{max} variable is reduced by 50% in Case 5 from Case 7, W_{eq} and t_b are significantly increased. This is likely due to the fact that the results converged such that M_o is about 53% when compared to the base case. Since a_{max} is reduced, less

propellant mass is needed. Therefore, M_P is reduced to about 50% while the L parameter is also reduced by about the same percentage as suggested by the L/D ratio resulting in a 73% lighter M_S when compared to the base case. This also results in a higher λ (+106%) and a reduced ε (-45%). Finally, in Case 6, a_{max} is increased by 100%. This results in a lower W_{eq} and t_b . However, the required M_P is significantly increased by about 102%, which also increases M_O of the configuration. The maximum λ of Case 6 is also roughly half when compared to the base case while ε is only reduced by about 3%. The rocket for this case also has an increased D and L when compared to the base case, which results in an M_S that is roughly double of what it is for Case 7.

OpenRocket Software Simulations

This part of the discussion section contains the results of the rocket that was modelled in the OpenRocket software based on the geometry parameters extracted from the MATLAB script for the base case (Case 7) that is highlighted in Tables 2 and 3. The results are as followed below.

I. X_{CG} & X_{CP} Location Verification

The exact geometry parameters (D and L) along with the wall thickness (δ), appropriate material properties, and payload mass (M_L) for the base case were used to model the sounding rocket in OpenRocket software. The motor was modelled in the Thrust Curve Tool (TCT) software with the predicted dimensional and mass values of the propellant from the MATLAB script and imported into OpenRocket. After all the components were modelled and imported in OpenRocket, it predicted the X_{CG} and X_{CP} locations of the model. These locations were compared with the predicted X_{CG} and X_{CP} locations from the MATLAB scripts (shown in Table 3 for Case 7). If the OpenRocket values were approximately within $\pm 0.02\text{m}$ of the obtained values from the MATLAB script, the X_{CG} and X_{CP} values were assumed to be correct. Figure 1 shows the X_{CG} and X_{CP} locations that were calculated by the OpenRocket software. These values are within $\pm 0.02\text{m}$ of what is highlighted in Table 3 for Case 7, therefore they were verified to be correct.

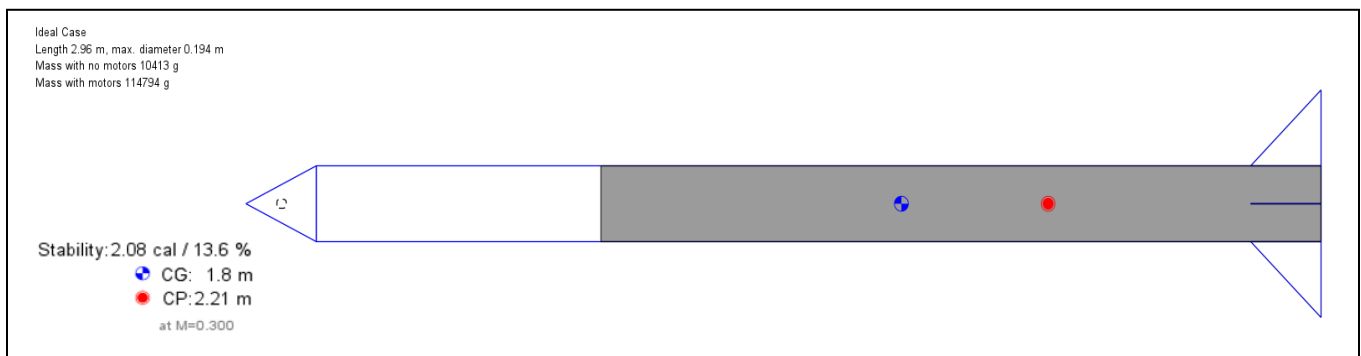


Figure 1: OpenRocket X_{CG} & X_{CP} Locations

II. Ideal Vertical Simulation

The rocket modelled part (I) was ‘flown’ in the ideal conditions (assuming no drag and vertical). This was done to compare the ideal results generated from the MATLAB script with the datapoints generated from the OpenRocket software. If the rocket ‘flown’ in OpenRocket achieved an apogee (h_{max}) that was within $\pm 2\text{m}$ of what was applied as a condition for the base case (Table 2), the design was further verified. Figure 2 shows the plot generated from the datapoints obtained from the OpenRocket software with an h_{max} , maximum vertical speed (V_{max}), and a_{max} at 6097m, 345 (m/s), and 9.96 respectively. These values seem to be very close to initial parameters from Table 2 for the base case.

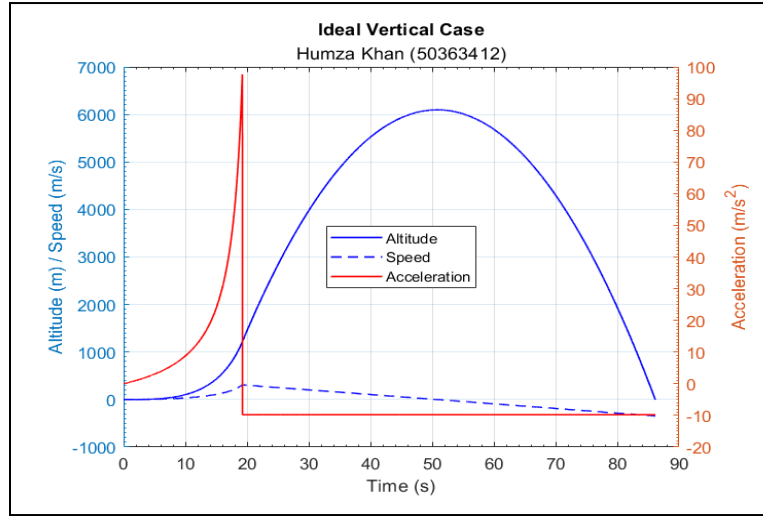


Figure 2: Ideal Vertical Simulation Plot

III. Drag Simulation (non-ideal)

The simulation in part (II) was modified to account for the drag and atmospheric conditions. Running this simulation resulted in a significantly reduced h_{max} , V_{max} , and a_{max} of about 2150m, 236 (m/s), and 6.59 respectively. This does not satisfy the base case conditions and it is due to the added resistance in motion caused by drag. The generated plot is shown below in Fig. 3.

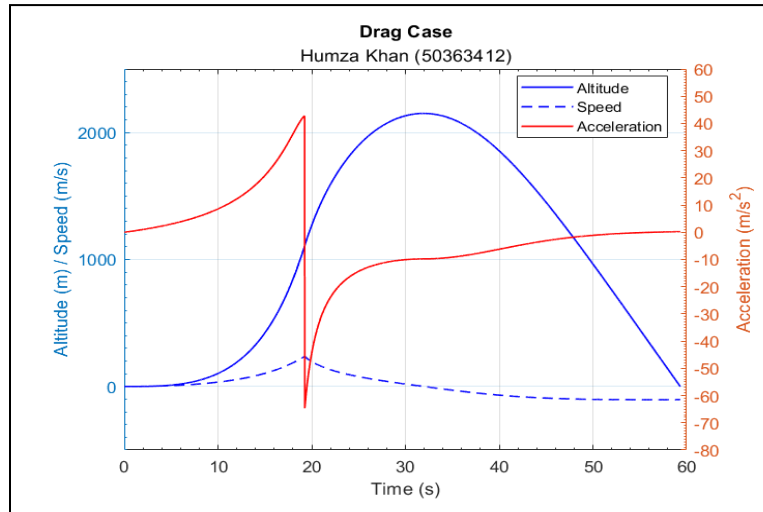


Figure 3: Drag Simulation Plot

IV. Changed Geometry Simulation (non-ideal)

Components such as the nose cone and fins were geometrically modified as shown in Fig. 4 (D and L parameters were kept the same) to be more aerodynamic and improve the apogee of the rocket as compared to part (III) simulation. The drag conditions were the same as part (III) and the simulation was run with the modified geometry (an ellipsoid nose cone, and curved fins were used to mitigate flow separation). The resulting h_{max} , V_{max} , and a_{max} of about 2328m, 245 (m/s), and 6.26 respectively, was a slight improvement from the previous simulation but the baseline conditions were not met. Figure 5 shows the plot for this simulation.

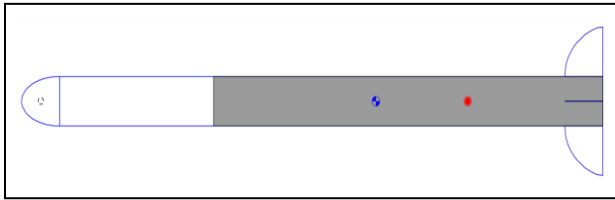


Figure 4: Modified Geometry

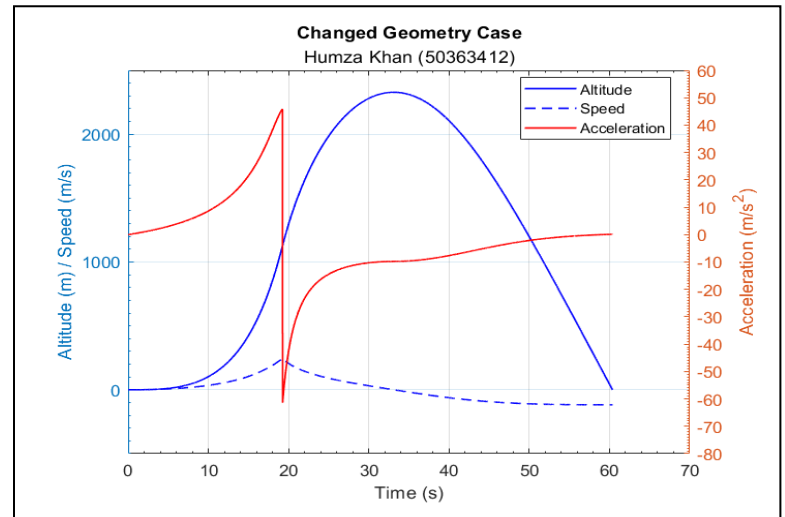


Figure 5: Changed Geometry Simulation Plot

V. Changed Material Simulation (non-ideal)

The geometry from part (IV) was kept the same, the only change was that a different material was selected. Aluminum was replaced by Carbon Fiber because it is a lighter material with a higher strength to weight ratio. The resulting h_{max} , V_{max} , and a_{max} from this simulation were found to be 2332m, 246 (m/s), and 6.4 respectively. This was a very slight improvement from the results of part (IV). Figure 6 shows the plot that was generated from the datapoints of this simulation.

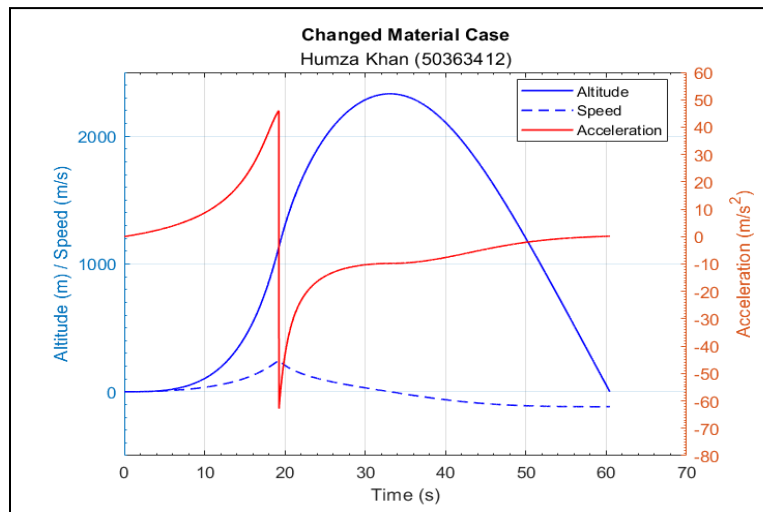


Figure 6: Changed Geometry Simulation Plot

VI. Modified Burn Profile Simulation

For this simulation, the same rocket was used as part (V)'s. The only thing that was modified was the motor burn profile. The TCT software was used to modify the motor in such a way that it has the same impulse, and dimensions as the motor used in other simulations, but this motor had an exponential decay with a higher peak thrust of 757lbs as compared to the previous motor that burned linearly with a peak thrust of 253lbs. The modified motor's burn profile is shown in Fig. 7, while the original motor's burn profile is shown in Fig. 8. The resulting h_{max} , V_{max} , and a_{max} from this simulation were found to be 3,194 m, 214 (m/s), and 2.4 respectively. Although, this was an improvement from the results of part (V), the results still did not achieve the baseline conditions. Figure 9 shows the plot that was generated from the datapoints of this simulation.

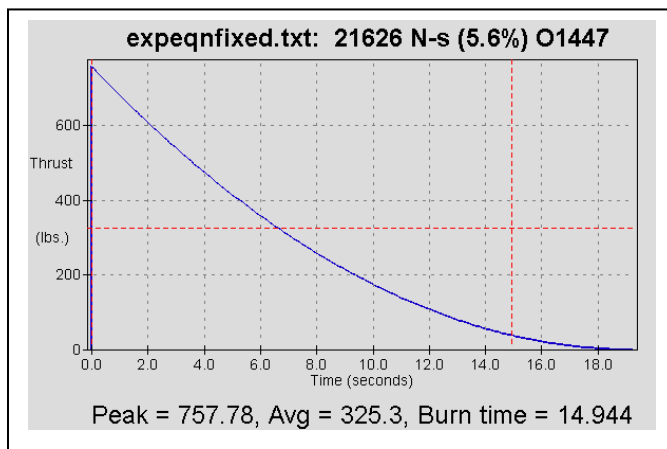


Figure 7: Modified Burn Profile

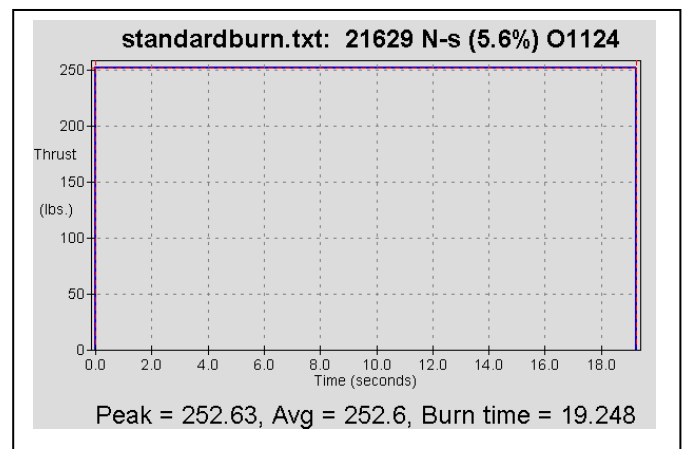


Figure 8: Original Burn Profile

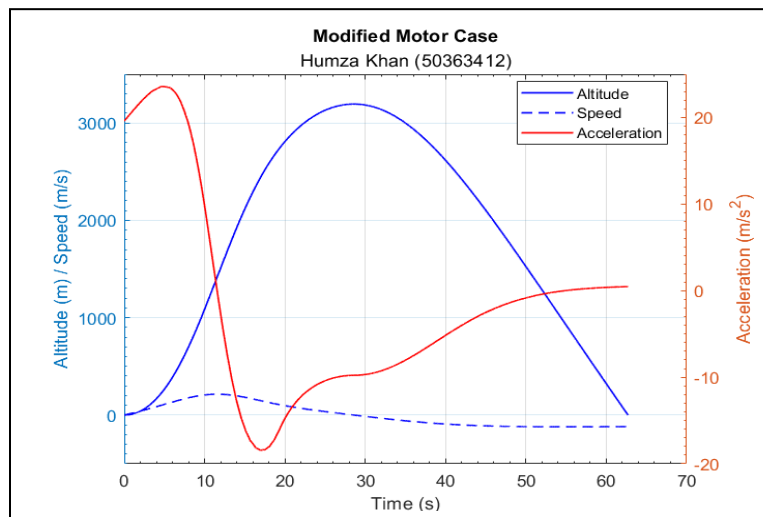


Figure 9: Modified Burn Profile Simulation Plot

Conclusion

The MATLAB script that was used to calculate the sounding rocket's geometry and other parameters was very accurate in predicting the rocket's performance under ideal conditions when drag effects are neglected. The limitation here is that the MATLAB script does not consider these neglected drag and aerodynamic effects which are later simulated via the OpenRocket software.

When drag effects are applied, the rocket configuration fails to reach the baseline requirement set for Case 7. Although aerodynamic improvements were made on the geometry of the components manually, the MATLAB code also does not take into account shape factors that can converge to a good aerodynamic performance as drag was neglected within the script.

Furthermore, the rocket structure is simplified a lot and assumes complete rigidity. The wall thickness (δ) obtained (Table 3) from the code is very thin, and not reasonable for a real-life construction at all. A rocket constructed with such δ value would just crumble under real environmental conditions and stresses.

In conclusion, MATLAB script is good for modelling ideal rockets under ideal conditions and software like OpenRocket and TCT are very good tools to verify their performance in not only ideal conditions, but also how differently they perform in real conditions as well.

Appendix A

MATLAB Script

```
%MAE 423 - Project Code
%Humza Khan (50363412)
clc; clear all; close all;

%% project starts here
%inputs
g = 9.81; %m/s
payload = [5, 10, 20]; %kg
h_max = [10000; 20000; 30000] / 3.281; %altitude converted to meters
a_max = [5, 10, 20]; %normalized acc
all_combos = [payload(1), h_max(2), a_max(2);
              payload(3), h_max(2), a_max(2);
              payload(2), h_max(1), a_max(2);
              payload(2), h_max(3), a_max(2);
              payload(2), h_max(2), a_max(1);
              payload(2), h_max(2), a_max(3);
              payload(2), h_max(2), a_max(2)]; %all possible inputs

SM = 2; %static margin
rho_s = 2700; %kg/m^3
rho_p = 1772; %kg/m^3
sigma_s = 150e6; %MPa ---> Pa
N = 4; %Number of Fins
R_opt = zeros(7,1); %initializing variable
W_eq = zeros(7,1); %initializing variable
t_burn = zeros(7,1); %initializing variable
mach = zeros(7,1); %initializing variable
[T_sl,a_sl,P_sl,rho_sl] = atmosisa(0);
T_stp = 298.15; %K Temp at STP
gamma = 1.4; %assuming diatomic atmosphere
R = 287; %universal gas constant
a_0 = sqrt(gamma.*T_stp.*R);

%=====
%% Solving for R_opt, W_eq, t_b, and Mach Numbers for each case
for i = 1:7
    R_opt(i) = 1 + all_combos(i,3);
    W_eq(i) = sqrt((all_combos(i,2).*g)./(0.5.*log(R_opt(i)).*(log(R_opt(i))-2)+((R_opt(i) - 1)./
R_opt(i))));
    t_burn(i) = ((R_opt(i) - 1)*W_eq(i))/(g*R_opt(i)); %optimal burn time eqn
    mach(i) = W_eq(i)/a_0; %solving for mach numbers @ SL
end
%=====
%% solving for gas dynamics stuff incl. total temp and pressure, mass, cg, cp
T_1 = T_stp.*(1+0.5.*(gamma-1).*mach.^2); %K total temp
P_0 = P_sl.*(T_1./T_stp).^(gamma/(gamma-1)); %Pa total pressure

guess_d = linspace(0,1,5000); %guess D range
guess_L = linspace(0,5,5000); %guess L range
%=====
%starting the loop
for k = 1:7
```

```

    delta = guess_d .* P_0(k,1) ./ (2 * sigma_s);
    L_D = guess_d + guess_L';
%=====
    %mass calculations
    %fin
    fin_sa = 0.5 .* guess_d .* guess_d; %SA of fin
    m_fin = N .* fin_sa .* delta .* rho_s; %mass of 4 fins
    m_fin_matrix = repmat(m_fin,[5000,1]); %matrix sizing
%=====
    %cone
    cone_sa = pi.*(0.5.*guess_d).*(sqrt( (0.5.*guess_d).^2 + guess_d.^2)); %SA of cone (no bottom
circle)
    m_cone = cone_sa .* delta .* rho_s; %mass of the cone
    m_cone_matrix = repmat(m_cone,[5000,1]);%matrix sizing
%=====
    %cylinder
    cylinder_sa = 2.*pi.*(L_D).*(0.5.*guess_d); %SA of cylinder (no top/bottom circle)
    m_cylinder = cylinder_sa .* delta .* rho_s; %no need to adjust sizing as dependant on both L
and D.
%=====
    %full structure
    m_structure = m_fin_matrix + m_cone_matrix + m_cylinder;
%=====
    %propellant mass
    m_p = (R_opt(k,1)-1) .* (m_structure + all_combos(k,1));
%=====
    %propellant length
    L_p = m_p ./ (guess_d.^2 .* (pi.*rho_p/4));
%=====
    %X.CG Calculation
    X.CG_nose = (2./3).*guess_d;
    X.CG_cylinder = guess_d + 0.5.*(L_D);
    X.CG_fins = (2./3).*guess_d + (L_D);
    X.CG_prop = (0.5.*L_p) + ((2.*guess_d + guess_L') - L_p);
    m_total = m_structure + m_p + all_combos(k,1);
    X.CG = ((X.CG_fins.*m_fin_matrix) + (X.CG_cylinder.*m_cylinder) + (X.CG_prop.*m_p) +
(X.CG_nose.*m_cone_matrix) + (X.CG_nose .* all_combos(k,1)))./m_total ;
%=====
    % X_CP calculation - Most Values were obtained from the Centuri Document
    X_conical = (2./3).*guess_d; %recalculating for sanity %center for normal force
    X_conical_matrix = repmat(X_conical,[5000,1]);
    a = guess_d;
    b = 0;
    S = guess_d;
    weird_l_length = sqrt(guess_d.^2 + (0.5.*guess_d).^2);
    CNaf = (4.*N.*(S./guess_d).^2)./(1 + sqrt(1 + (2.*weird_l_length./(a + b)).^2)); %found this
to be constant throughout
    R = 0.5.*guess_d; %radius
    kfb = 1 + (R./(S+R)); %interference factor for 4 fins
    CNafb = kfb.*CNaf; %normal force on fins in presence of the body
    CNan = 2; %normal force on the nose
    x_f = guess_d + guess_L';
    m = guess_d;
    del_xf = (((m.*(a + 2.*b))./(3.*(a + b)))) + (1/6).*(a + b - (a.*b)./(a+b));
    xf_bar = x_f + repmat(del_xf,[5000,1]); %total center of pressure in inches
    CNa = CNafb + CNan; %total norm coeff
    %solving for x_cp

```

```

X_CP = (CNan.*X_conical_matrix + repmat(CNafb,[5000,1]).*xf_bar)./CNa; %center of pressure
location from the top of the rocket
%=====
lamda = all_combos(k,1)./(m_structure+m_p);

%stability analysis
const_1 = abs((X_CP - X_CG) ./ repmat(guess_d,[5000,1]));
const_1norm = abs(const_1) - 2; %enforces SM normalizes
const_1norm1 = abs(const_1norm);

min_const1(1,k) = min(const_1norm1(:));
[min_row, min_col] = find(const_1norm1 == min_const1(1,k));

const_2 = lamda;

max_lamda(1,k) = const_2(min_row, min_col);
[max_row, max_col] = find(const_2 == max_lamda(1,k));

%=====
%% Extracting Case-By-Case Values in this section
P0_fraction(1,k) = P_0(k,1)/P_sl;
dD_fraction(1,k) = delta(1,max_col)/guess_d(1,max_col);
final_d(1,k) = guess_d(1,max_col);
final_L(1,k) = guess_L(1,max_row);
LD_fraction(1,k) = final_L(1,k)/final_d(1,k);
final_XCG(1,k) = X_CG(max_row,max_col);
final_XCP(1,k) = X_CP(max_row,max_col);
final_mp(1,k) = m_p(max_row,max_col);
final_ms(1,k) = m_structure(max_row,max_col);
final_m0(1,k) = m_total(max_row,max_col);
final_lamda(1,k) = max_lamda(1,k);
epsilon(1,k) = m_structure(max_row,max_col)/(m_structure(max_row,max_col) +
m_p(max_row,max_col));

%=====

end
%thrust = m_p.*W_eq./t_burn; ----> Thrust eqn used.
%=====

%% organizing data in arrays
Cases = all_combos;
dataset_extract = [R_opt, W_eq, t_burn, P0_fraction', dD_fraction',final_d', final_L',
LD_fraction', final_XCG', final_XCP', final_mp', final_ms', final_m0', final_lamda',epsilon'];
FINAL_TABLE = [Cases, dataset_extract]; %final answer that was used to form the table.
%=====

```